

SPRING FRAMEWORK

Content

- What is Spring Framework?
- Key features of Spring Framework
 - Dependency Injection and Inversion of Control
 - Aspect Oriented Programming
 - Spring Modules
- Advantages of using Spring Framework

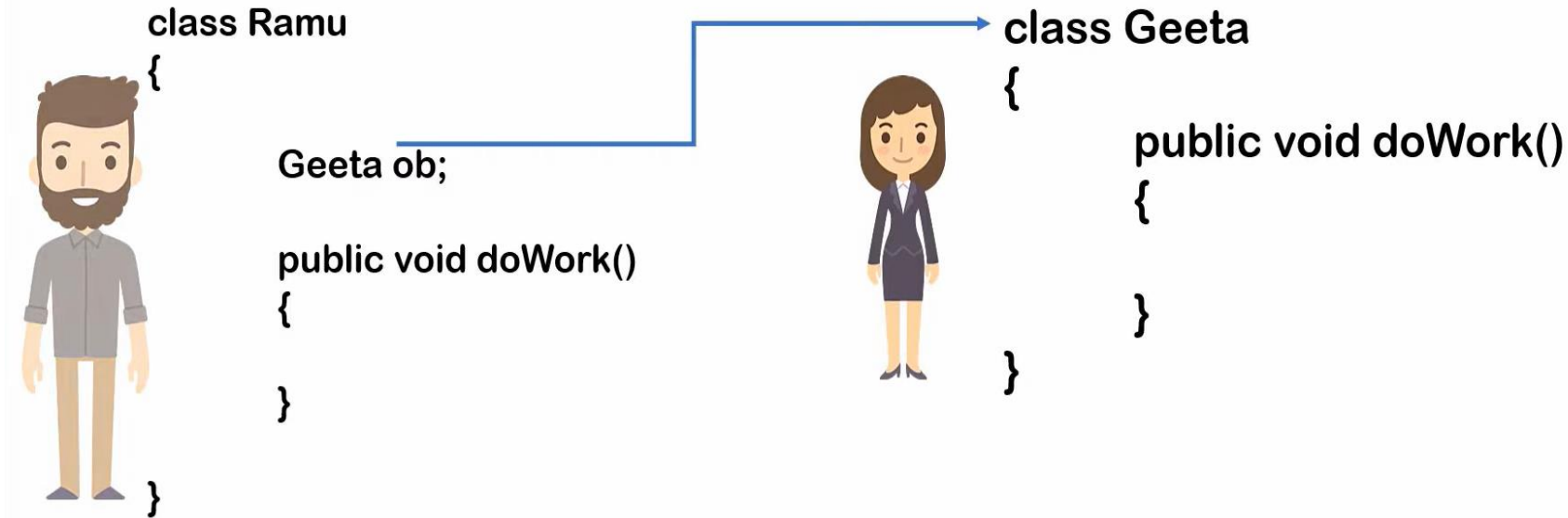


What is Spring?

- Spring is a Dependency Injection framework to make java application loosely coupled
 - Easy to change
- Spring Framework makes the easy development of JavaEE application
- Most popular Java framework

Dependency Injection

It is design pattern

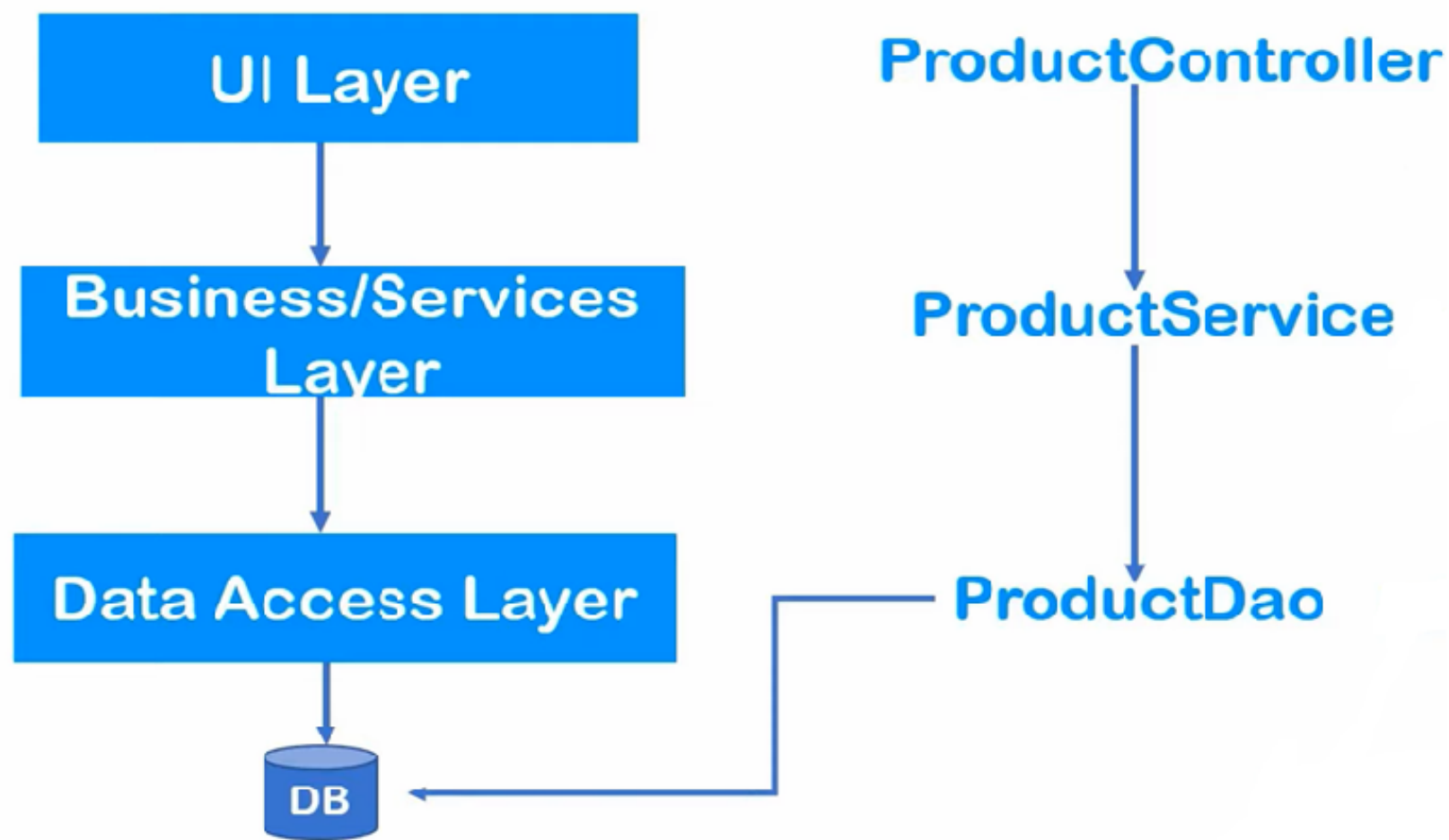


- Create object of geeta using new
- It makes Tight coupling between these two classes.
- It means that changes on geeta may required changes in Ramu

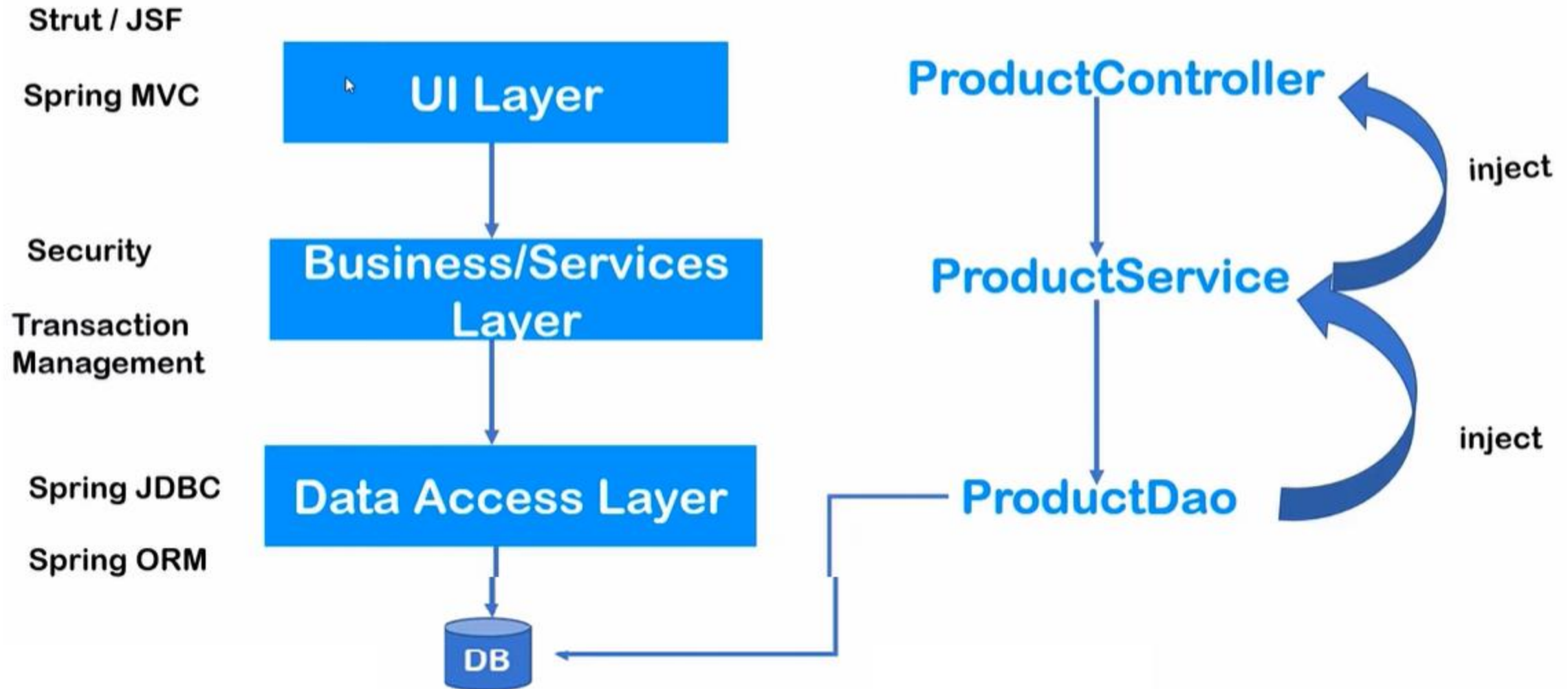
Dependency Injection

- Spring create object of Geeta and inject into Ramu
- Like doctor inject medicine to human being by means of injection
- Now designing is loosely coupled
- This complete process is referred as **Inversion of Control (IoC)**
- But my some how we have tell the IoC container about to create an object of Geeta
 - This can be done by metadata
 - XML file
 - Annotations

Spring and JEE



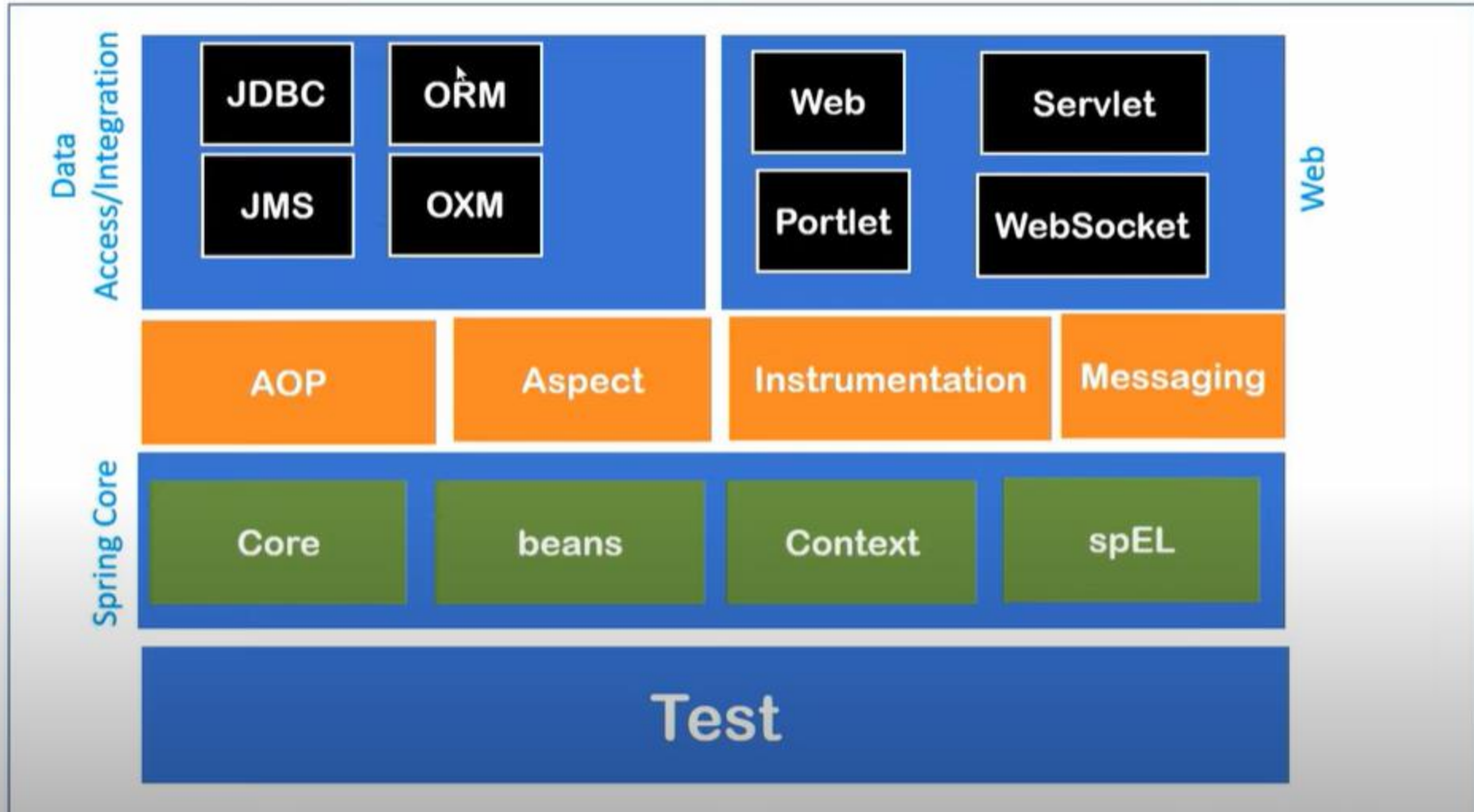
Spring and JEE



Spring Modules



Spring Framework





What is Spring Framework?

- Spring is **the most popular application development framework** for **enterprise** Java.
- Open source Java platform since 2003.
- Spring supports all major application servers and JEE standards.
- Spring handles the infrastructure so you can focus on your application.



Dependency Injection

Introduction to Concept

- The technology that actually defines Spring (Heart of Spring).
- Dependency Injection helps us to keep our classes as independent as possible.
 - Increase reuse by applying low coupling
 - Easy testing
 - More understandable



Dependency Injection

Introduction to Concept

An injection is the passing of a dependency (a service) to a dependent object (a client). Passing the service to the client, rather than allowing a client to build or find the service, is the fundamental requirement of the pattern.

“Dependency injection is a pattern where the container passes objects by name to other objects, via either constructors, properties, or factory methods.”



Dependency Injection

Relationship Between DI and Inversion of Control

In software engineering, inversion of control (IoC) describes a design in which custom-written portions of a computer program receive the flow of control from a generic, reusable library.

The Inversion of Control (IoC) is a general concept, and it can be expressed in many different ways and dependency Injection is merely one concrete example of Inversion of Control.



Dependency Injection

IoC Container

- The Spring container (IoC Container) is at the core of the Spring Framework.
- The container will create the objects, wire them together, configure them, and manage their complete lifecycle from creation till destruction.

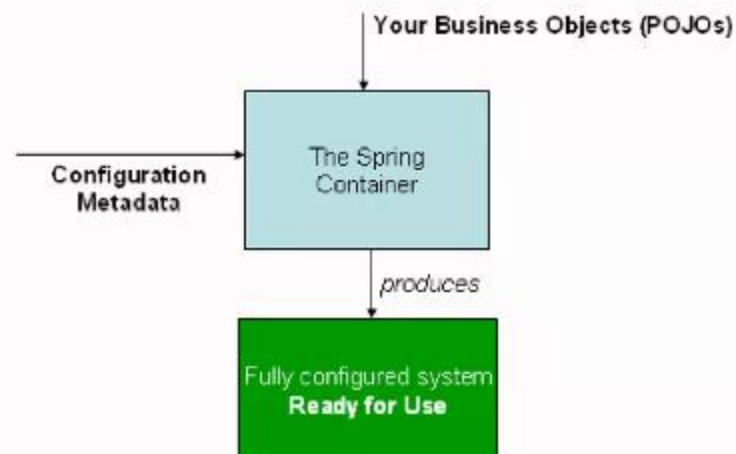




Dependency Injection

IoC Container

- The container gets its instructions on what objects to instantiate, configure, and assemble by reading configuration metadata provided.
- The configuration metadata can be represented either by;
 - XML,
 - Java annotations,
 - Java code.





Dependency Injection

Code Example

```
1 package basicinjection;
2
3 public class Foo {
4     private String name;
5
6     public Foo() { }
7
8     public Foo(String name) {
9         this.name = name;
10    }
11
12    public void setName(String name) {
13        this.name = name;
14    }
15
16    public String getName() {
17        return name;
18    }
19 }
20 }
```

```
1 package basicinjection;
2
3 public class Bar {
4     private String name;
5     private int age;
6     private Foo foo;
7
8     public Bar() {}
9
10    public Bar(String name, int age) {
11        this.name = name;
12        this.age = age;
13    }
14    public void setFoo(Foo foo) {
15        this.foo = foo;
16    }
17
18    public void processFooName() {
19        System.out.println("Name in Injected Foo is: "+foo.getName());
20    }
21
22    @Override
23    public String toString() {
24        return "Bar has name = "+this.name+" and age = "+this.age;
25    }
26 }
```

To instantiate the above classes, one way is to do the usual new operator like `new Foo()` or `new Bar()` OR we can use the Spring dependency injection to instantiate these classes and set the properties accordingly.



Dependency Injection

Code Example

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
```

```
  <bean id="foo" class="basicinjection.Foo" scope="prototype">
    <constructor-arg index="0" value="Cleopatra"></constructor-arg>
```

→ **Foo f = new Foo("Cleopatra");**

```
  </bean>
```

```
  <bean id="bar" class="basicinjection.Bar">
```

```
    <constructor-arg type="int" value="26" />
```

```
    <constructor-arg type="java.lang.String" value="Arthur" />
```

```
    <property name="foo" ref="foo"></property>
```

→ **Bar b = new Bar("Arthur",26);
b.setFoo(f);**

```
  </bean>
```

```
</beans>
```



Dependency Injection

Code Example

```
1 package basicinjection;
2
3 import org.springframework.context.support.ClassPathXmlApplicationContext;
4
5 public class TestFooBar {
6
7     public static void main(String[] args) throws InterruptedException {
8         ClassPathXmlApplicationContext applicationContext = new ClassPathXmlApplicationContext(
9             "basicinjection/springbasic.xml");
10        Bar bar = applicationContext.getBean("bar", Bar.class);
11        bar.processFooName();
12        System.out.println(bar.toString());
13        /*
14         * if a single definition of a class type exists, then u can get the
15         * instance by this way also. No need to specify Id
16         */
17        Foo foo = applicationContext.getBean(Foo.class);
18        System.out.println(foo.getName());
19    }
20
21 }
```

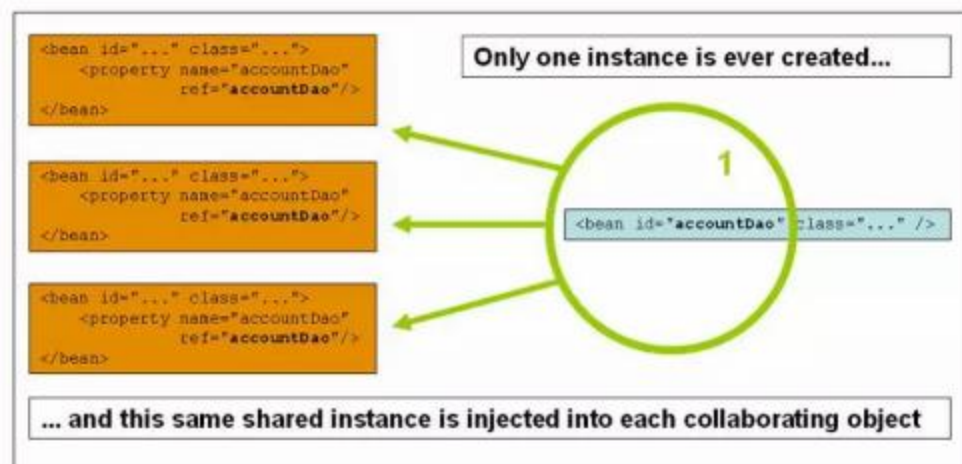
Spring's `ClassPathXmlApplicationContext` is the commonly used object that hold the information of all the beans that it instantiates.



Dependency Injection

Bean Scopes

Scope	Description
<u>Singleton</u>	(Default) Scopes a single bean definition to a single object instance per Spring IoC container.
<u>Prototype</u>	Scopes a single bean definition to any number of object instances.





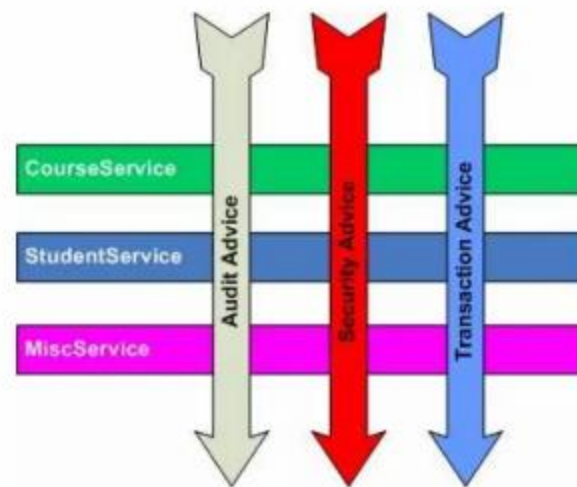
Aspect Oriented Programming (AOP)

Introduction to Concept

- AOP entails breaking down program logic into distinct parts called **concerns**.
- The functions that span multiple points of an application are called **cross-cutting concerns** and these cross-cutting concerns are conceptually separate from the application's business logic.
- AOP is like triggers in programming languages such as Perl, .NET, Java and others.

Examples of cross-cutting concerns:

- Logging
- Security
- Transaction
- Caching





Aspect Oriented Programming (AOP)

Spring AOP

- Spring AOP module provides interceptors to intercept an application, for example, when a method is executed, you can add extra functionality before or after the method execution.
- Spring AOP's approach to AOP differs from that of most other AOP frameworks. The aim is to provide a close integration between AOP implementation and Spring IoC, not to provide the most complete AOP implementation.
- Spring Framework's AOP functionality is normally used in conjunction with the Spring IoC container. Aspects are configured using normal bean definition syntax.



Aspect Oriented Programming (AOP)

Code Example

File : Spring-Customer.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:aop="http://www.springframework.org/schema/aop"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop-3.0.xsd ">

    <aop:aspectj-autoproxy />

    <bean id="customerBo" class="com.mkyong.customer.bo.impl.CustomerBoImpl" />

    <!-- Aspect -->
    <bean id="logAspect" class="com.mkyong.aspect.LoggingAspect" />

</beans>
```



Aspect Oriented Programming (AOP)

Code Example

File : LoggingAspect.java

```
package com.mkyong.aspect;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;

@Aspect
public class LoggingAspect {

    @Before("execution(* com.mkyong.customer.bo.CustomerBo.addCustomer(..))")
    public void logBefore(JoinPoint joinPoint) {

        System.out.println("logBefore() is running!");
        System.out.println("hijacked : " + joinPoint.getSignature().getName());
        System.out.println("*****");

    }

}
```

Pointcut (AspectJ Pointcut Expression Language)

Join Point

Advice

- **@Before** – Run before the method execution
- **@After** – Run after the method returned a result
- **@AfterReturning** – Run after the method returned a result, intercept the returned result as well.
- **@Around** – Run around the method execution, combine all three advices above.
- **@AfterThrowing** – Run after the method throws an exception



Aspect Oriented Programming (AOP)

Code Example

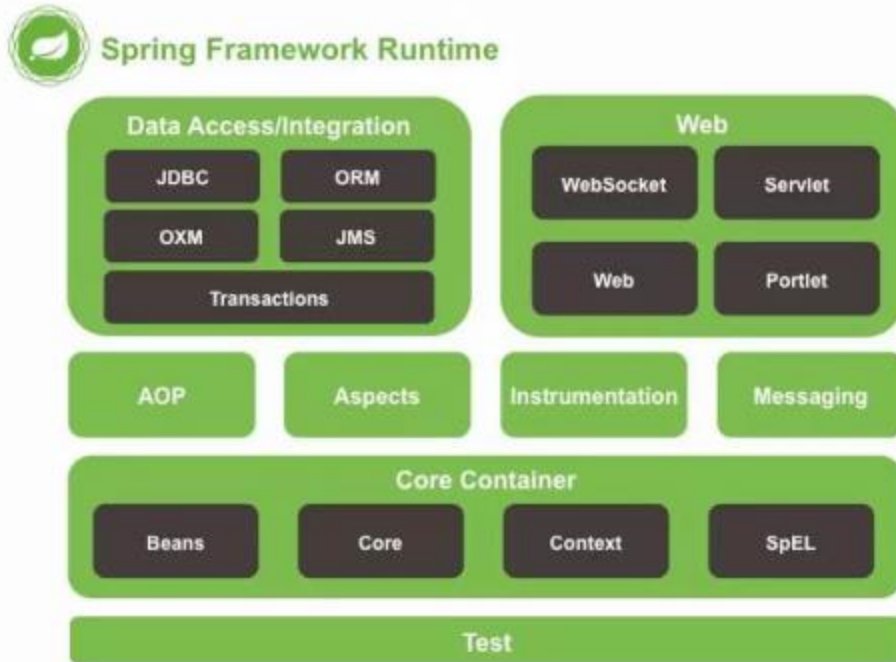
Run it

```
CustomerBo customer = (CustomerBo) appContext.getBean("customerBo");  
customer.addCustomer();
```

Output

```
logBefore() is running!  
hijacked : addCustomer  
*****  
addCustomer() is running
```

Spring Modules



Overview of the Spring Framework

- The Spring Framework consists of features organized into about 20 modules.
- These modules are grouped into Core Container, Data Access/Integration, Web, AOP (Aspect Oriented Programming), Instrumentation, and Test.

Advantages of Using Spring Framework

- Open source
- Lightweight and fast
- Modular structure
- Low coupling thanks to Dependency Injection
- Resuable software
- AOP support
- Stable and lots of resources
- Projects that make our life easier like Spring Security



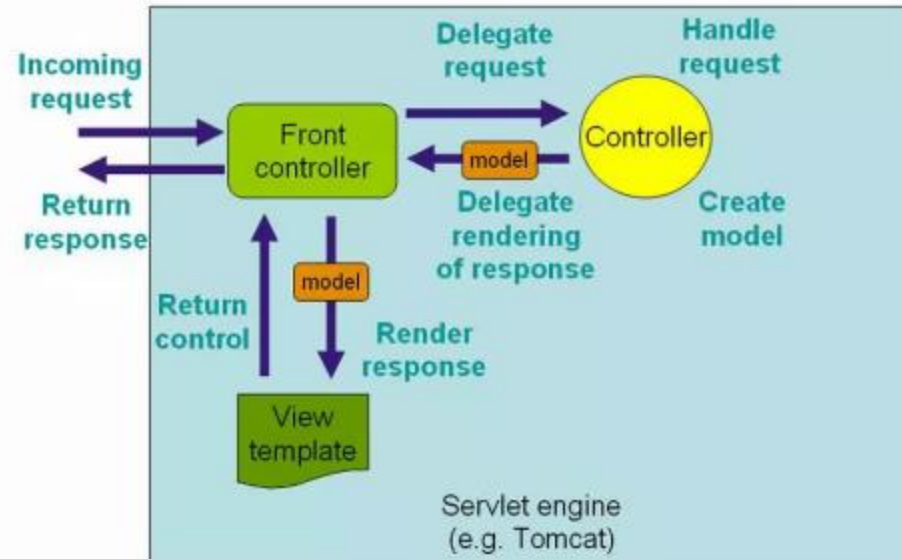
Content

- Spring MVC
- Spring RESTful Services
- Spring Security
- Spring Test



Spring MVC

- The Spring web MVC framework provides model-view-controller architecture and ready components that can be used to develop flexible and loosely coupled web applications.
- The MVC pattern results in separating the different aspects of the application (input logic, business logic, and UI logic), while providing a loose coupling between these elements.





Spring MVC

MVC Bean Scopes

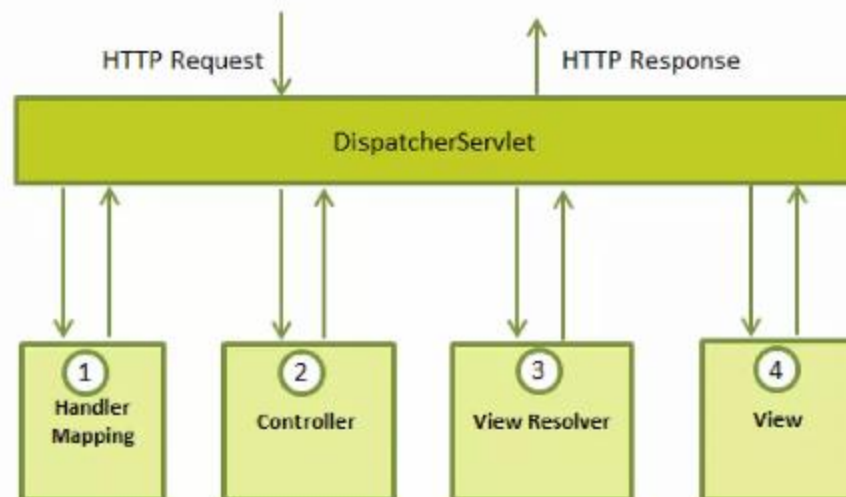
Scope	Description
<u>Request</u>	Scopes a single bean definition to the lifecycle of a single HTTP request; that is, each HTTP request has its own instance of a bean created off the back of a single bean definition. Only valid in the context of a web-aware Spring ApplicationContext.
<u>Session</u>	Scopes a single bean definition to the lifecycle of an HTTP Session. Only valid in the context of a web-aware Spring ApplicationContext.
<u>Global Session</u>	Scopes a single bean definition to the lifecycle of a global HTTP Session. Typically only valid when used in a portlet context. Only valid in the context of a web-aware SpringApplicationContext.
<u>Application</u>	Scopes a single bean definition to the lifecycle of a ServletContext. Only valid in the context of a web-aware Spring ApplicationContext.



Spring MVC

The DispatcherServlet

- The Spring Web model-view-controller (MVC) framework is designed around a *DispatcherServlet* that handles all the HTTP requests and responses.





Spring MVC

Web.xml

- You need to map requests that you want the *DispatcherServlet* to handle, by using a URL mapping in the **web.xml** file.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app>

<display-name>Example</display-name>

<context:component-scan base-package="com.proj.web" />

<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>
        /WEB-INF/context/example-general-context.xml
    </param-value>
</context-param>

<listener>
    <listener-class>
        org.springframework.web.context.
        ContextLoaderListener
    </listener-class>
</listener>
```

```
<servlet>
    <servlet-name>spring</servlet-name>
    <servlet-class>
        org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/example-servlet.xml</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
    <servlet-name>spring</servlet-name>
    <url-pattern>/</url-pattern>
</servlet-mapping>

</web-app>
```



Spring MVC

Servlet.xml

- Now, let us check the required configuration for **example-servlet.xml** file, placed in your web application's *WebContent/WEB-INF* directory:

```
<context:component-scan base-package="com.proj.web.controller" />

<bean id="viewResolver" class="org.springframework.web.servlet.view.UrlBasedViewResolver">
    <property name="viewClass" value="org.springframework.web.servlet.view.JstlView" />
    <property name="prefix" value="/WEB-INF/view/" />
    <property name="suffix" value=".jsp" />
</bean>

<mvc:annotation-driven />

</beans>
```



Spring MVC

Controller & View

- HomeController.java:

```
@Controller
public class HomeController {

    @RequestMapping(value = { "/", "/home" },
        method = RequestMethod.GET)

    public String showHomePage(ModelMap model) {

        model.addAttribute("message", "Hello
        Spring MVC Framework!");

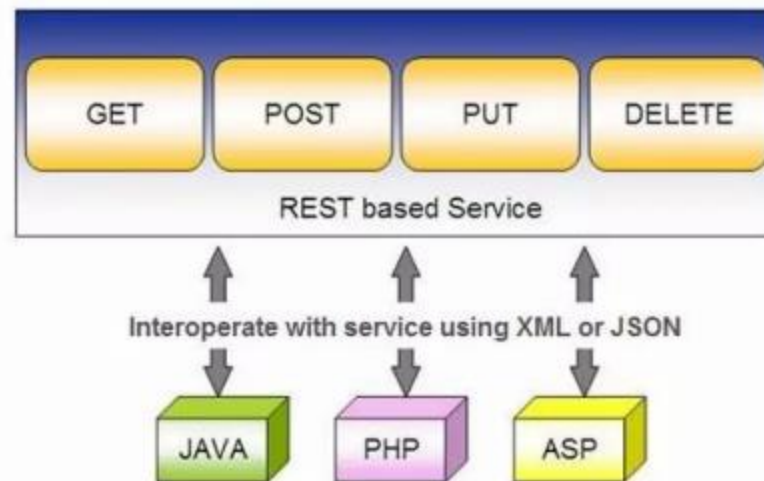
        return "home";
    }
}
```

- Home.jsp

```
<html>
<head>
<title>Here is home page</title>
</head>
<body>
<h2>${message}</h2>
</body>
</html>
```

Spring RESTful Services

- *REST* does not require the client to know anything about the structure of the API. Rather, the server needs to provide whatever information the client needs to interact with the service.



Spring RESTful Services

- Spring's annotation-based MVC framework serves as the basis for creating RESTful Web Services.
- RESTful services use URIs to name resources. To facilitate accessing the information contained in a URI, its structure follows conventions so that it can easily be described in a parameterized form.

```
http://www.example.com/users/{userid}
```

contains the variable 'userid'. If we assign the variable the value "fred", then 'expanding' the URI Template gives.

```
http://www.example.com/users/fred
```

Spring RESTful Services

- Spring uses the `@RequestMapping` method annotation to define the URI Template for the request. The `@PathVariable` annotation is used to extract the value of the template variables and assign their value to a method variable.

```
1 @RequestMapping(value = "/users/{userId}", method = RequestMethod.GET, produces = "application/json")
2 public @ResponseBody User findUser(@PathVariable String userId) {
3
4     User user = userService.findUser(userId);
5
6     return user;
7 }
```

- When a request comes in for `/users/serhat`, the value `'serhat'` is bound to the method parameter `String userId`.

Spring RESTful Services

- A RESTful architecture may expose multiple representations of a resource. There are two strategies for a client to inform the server of the representation it is interested in receiving.
- The first strategy is to use a distinct URI for each resource. This is typically done by using a different file extension in the URI.
 - For example the URI `http://www.example.com/users/serhat.pdf` requests a PDF representation of the user `serhat` while `http://www.example.com/users/serhat.xml` requests an XML representation.
- The second strategy is for the client to use the same URI to locate the resource but set the Accept HTTP request header to list the media types that it understands.
 - For example, a HTTP request for `http://www.example.com/users/serhat` with an Accept header set to `application/pdf` requests a PDF representation of the user `serhat` while `http://www.example.com/users/serhat` with an Accept header set to `text/xml` requests an XML representation. This strategy is known as **content negotiation**.



Spring Security



Spring Security is a framework that focuses on providing both authentication and authorization to Java applications.

Features:

- Comprehensive and extensible support for both Authentication and Authorization
- Protection against attacks like session fixation, clickjacking, cross site request forgery, etc
- Servlet API integration
- Optional integration with Spring Web MVC
- Much more...

Spring Test

Spring Test Framework supports;

- Unit testing with mock objects
- Easy unit testing for Controllers
- IoC container to create dependencies for Integration Testing
- Transaction management for Integration Testing
- Third party frameworks like JUnit, TestNG, Mockito



